

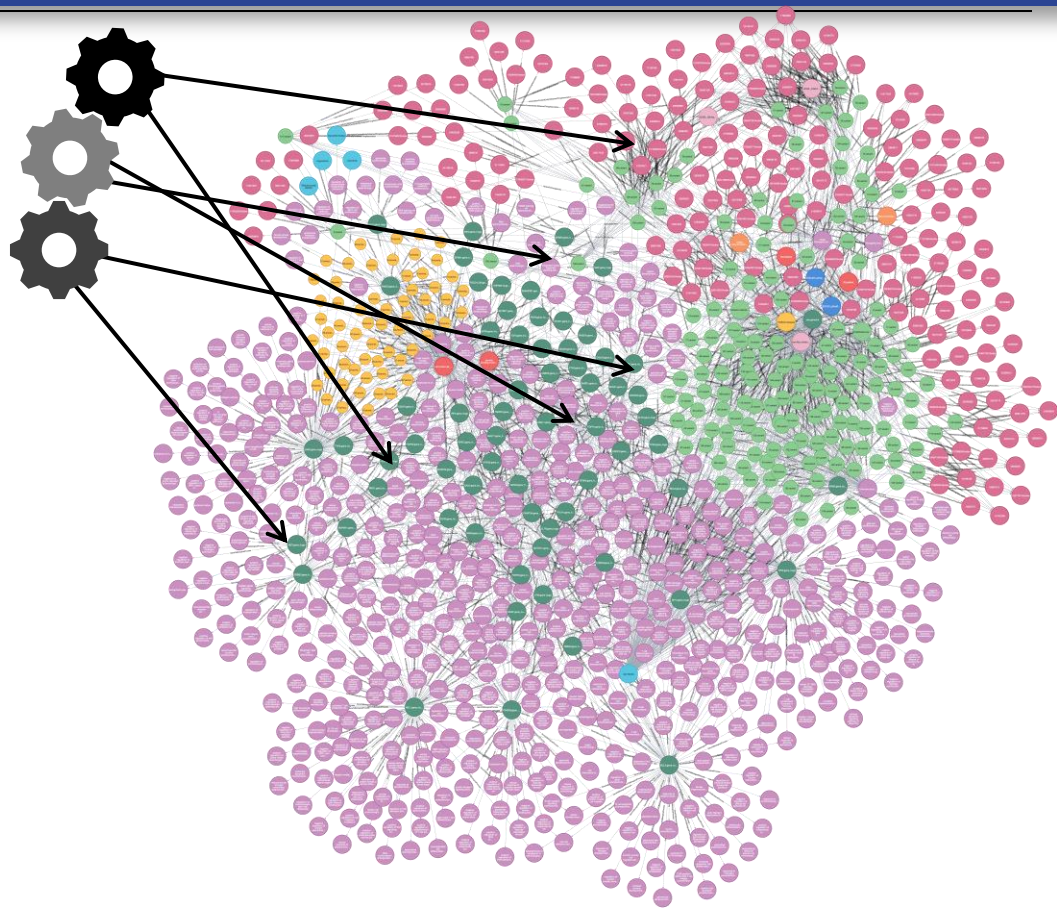
Hands on OntoWeaver

Extract, Transform, *Fuse*, Load (into BioCypher)
without much **code**

Johann Dreo, Claire Laudy, Matthieu Najm, Benno Schwikowski
Computational Systems Biomedicine Lab
Institut Pasteur

FAIR SKG building

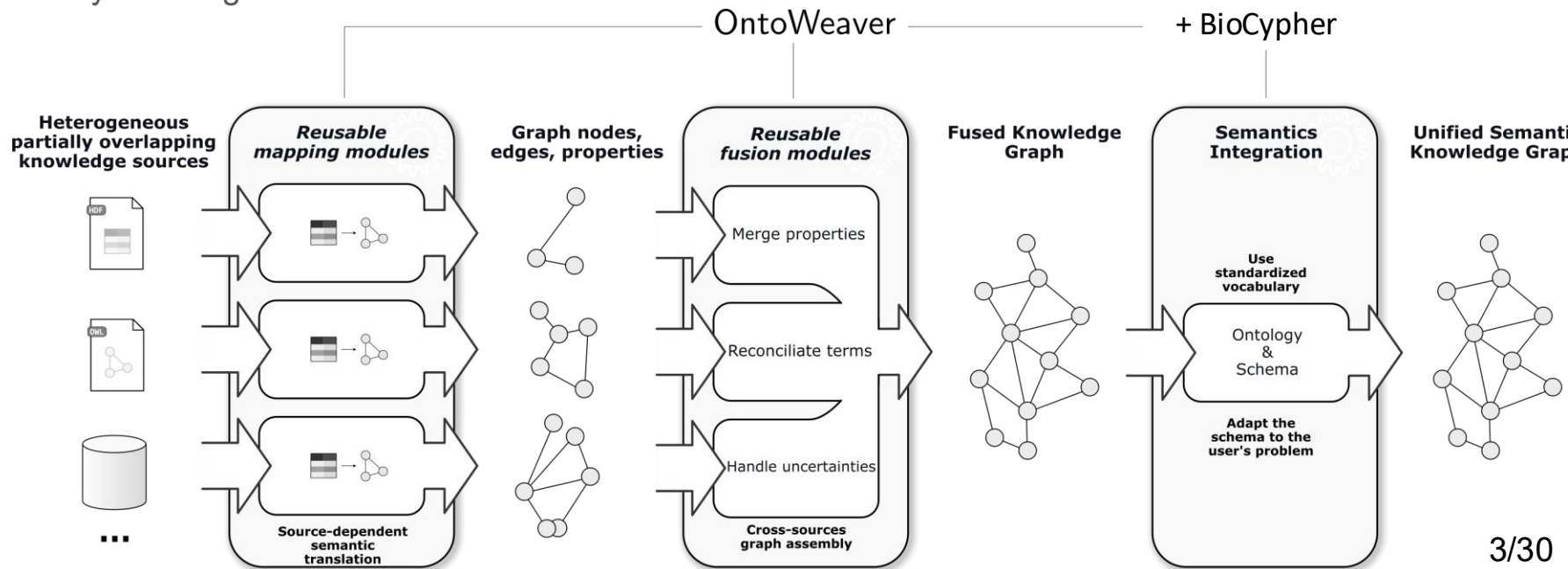
- Automated “adapters” scripts = **modular building blocks**:
 - Findable
 - Accessible
 - Interoperable
 - Reproducible
- Make a tailored SKG = enable a set of adapters.



Biomedical Knowledge Graph Integration is Hard

We want *ETL modules* that:

- extract data from a (structured = table or queriable doc) source
- are reusable, so as to *compose* a new graph
- are easy to configure



Alternative approaches

- “Raw” BioCypher adapter
 - Fully in charge of the ETL code and the fusion problem
- ETL code from BioCroissant Baker
 - in Croissant: columns semantics
 - = OntoWeaver *explicit* mapping
 - Fusion?

Installation & test

1. Install <https://docs.astral.sh/uv>
2. git clone <https://github.com/oncodash/ontoweaver.git>
3. cd ontoweaver
4. uv run ontoweave --help
5. cd tests/demo
6. uv run ontoweave -C config.yaml -s schema.yaml data.csv:mapping.yaml
7. ./make_doc.sh
8. Browse: ontoweaver/doc_built/index.html

The ontoweave command

```
$ ontoweave  
  | data.csv:mapping.yaml  
  | --biocypher-config config.yaml  
  | --biocypher-schema schema.yaml  
  
  | --debug
```

YAML mapping DSL

```
row:  
  <options>  
  
metadata:  
  <properties>  
  
transformers:  
  - <name>:  
    <options>  
  - <name>:  
    <options>  
  - ...
```

Given a table, ...

	patient_id	referenceGenome	dataVersion	citationPMids	
1	P2	RG1	v3.14	10	11
2	P1	RG1	v3.14	11	12

row:

rowIndex:

to_subject: **variant**

transformers:

- map:

column: **patient_id**

to_object: **patient**

via_relation: **patient_has_variant**

Given a table, ...

	patient_id	referenceGenome	dataVersion	citationPMids	
1	P2	RG1	v3.14	10	11
2	P1	RG1	v3.14	11	12

row:

rowIndex:

to_subject: **variant**

metadata:

source: "OntoWeaver"

transformers:

- **map:**

column: **patient_id**

to_object: **patient**

via_relation: **patient_has_variant**

- **split:**

column: **citationPMids**

to_object: **publication**

via_relation: **reference_genome**

separator: ","

- **map:**

column: **dataVersion**

to_property: **version**

for_objects:

- **genome**

.. map columns to nodes, ...

	patient_id	referenceGenome	dataVersion	citationPMids
1	patient P2	genome RG1	v3.14	10 11
2	P1	RG1	v3.14	11 12

row:

rowIndex:

to_subject: variant

metadata:

source: "OntoWeaver"

transformers:

- **map:**

column: patient_id

to_object: patient

via_relation: patient_has_variant

- **split:**

column: citationPMids

to_object: publication

via_relation: reference_genome

separator: ","

- **map:**

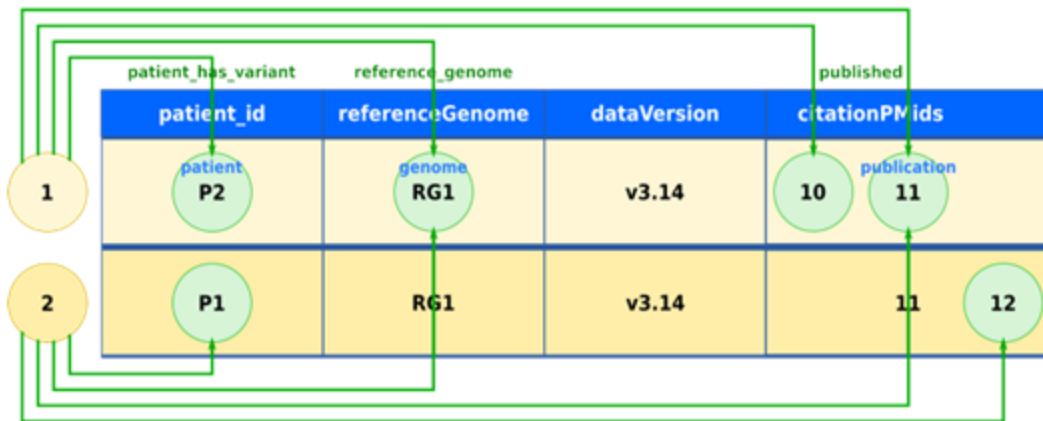
column: dataVersion

to_property: version

for_objects:

- genome

... link subject and nodes with edges, ...



row.

rowIndex:

to subject: variant

metadata:

source: "OntoWeaver"

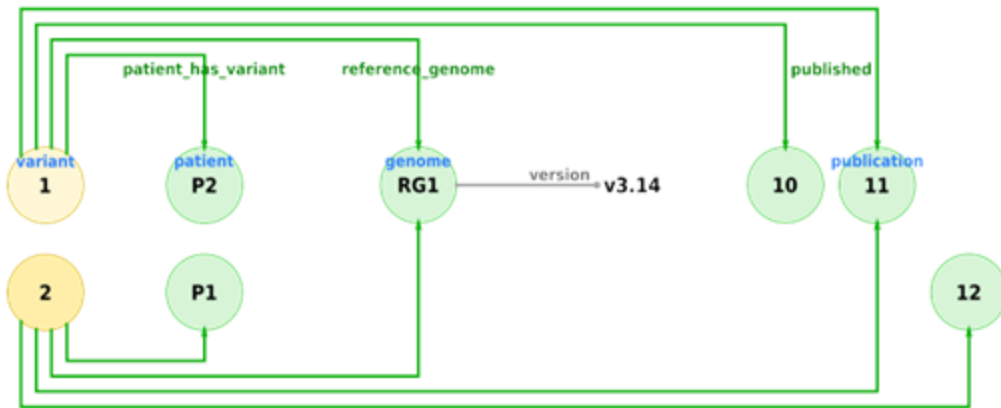
transformers:

- ```
- map:
 column: patient_id
 to_object: patient
 via_relation: patient_has_variant

- split:
 column: citationPMids
 to_object: publication
 via_relation: reference_genome
 separator: ","

- map:
 column: dataVersion
 to_property: version
 for_objects:
 - genome
```

... you have a typed graph.



*row:*

**rowIndex:**

to\_subject: **variant**

*metadata:*

source: "OntoWeaver"

*transformers:*

- **map:**

column: **patient\_id**

to\_object: **patient**

via\_relation: **patient\_has\_variant**

- **split:**

column: **citationPMids**

to\_object: **publication**

via\_relation: **reference\_genome**

separator: ",",

- **map:**

column: **dataVersion**

to\_property: **version**

for\_objects:

- **genome**

# Transformers

[https://ontoweaver.readthedocs.io/en/latest/sections/mapping\\_api.html#available-transformers](https://ontoweaver.readthedocs.io/en/latest/sections/mapping_api.html#available-transformers)

## Writing a Mapping

How are config files related?

### Available Transformers

map

split

cat

cat\_format

nested

split\_nested

string

translate

replace

boolean

maths

Case manipulation transformers

Advanced type management

Keyword Synonyms

# Structured documents = queries [on iterable]

## *JSON*

```
{
 "data": [
 {"variant": 0, "patient": "A", "age": 12 },
 {"variant": 1, "patient": "B", "age": 23 },
 {"variant": 2, "patient": "C", "age": 34 }
]
}
```

## *row:*

### *map:*

**element:** data[\*].variant

to\_subject: variant

### *transformers:*

#### *- map:*

**element:** data[\*].patient

to\_object: patient

via\_relation: patient\_has\_variant

#### *- map:*

**element:** data[\*].age

to\_property: age

for\_object: patient

# Why a declarative mapping?

**subject:** variant

columns:

**patient\_id:**

to\_object: patient

via\_relation: patient\_has\_variant

referenceGenome:

to\_object: genome

via\_relation: reference\_genome

dataVersion:

to\_property:

version:

- genome

**citationPMids:**

into transformer:

split:

```
separator: ","
```

to\_object: publication

via relation: published



# Try it

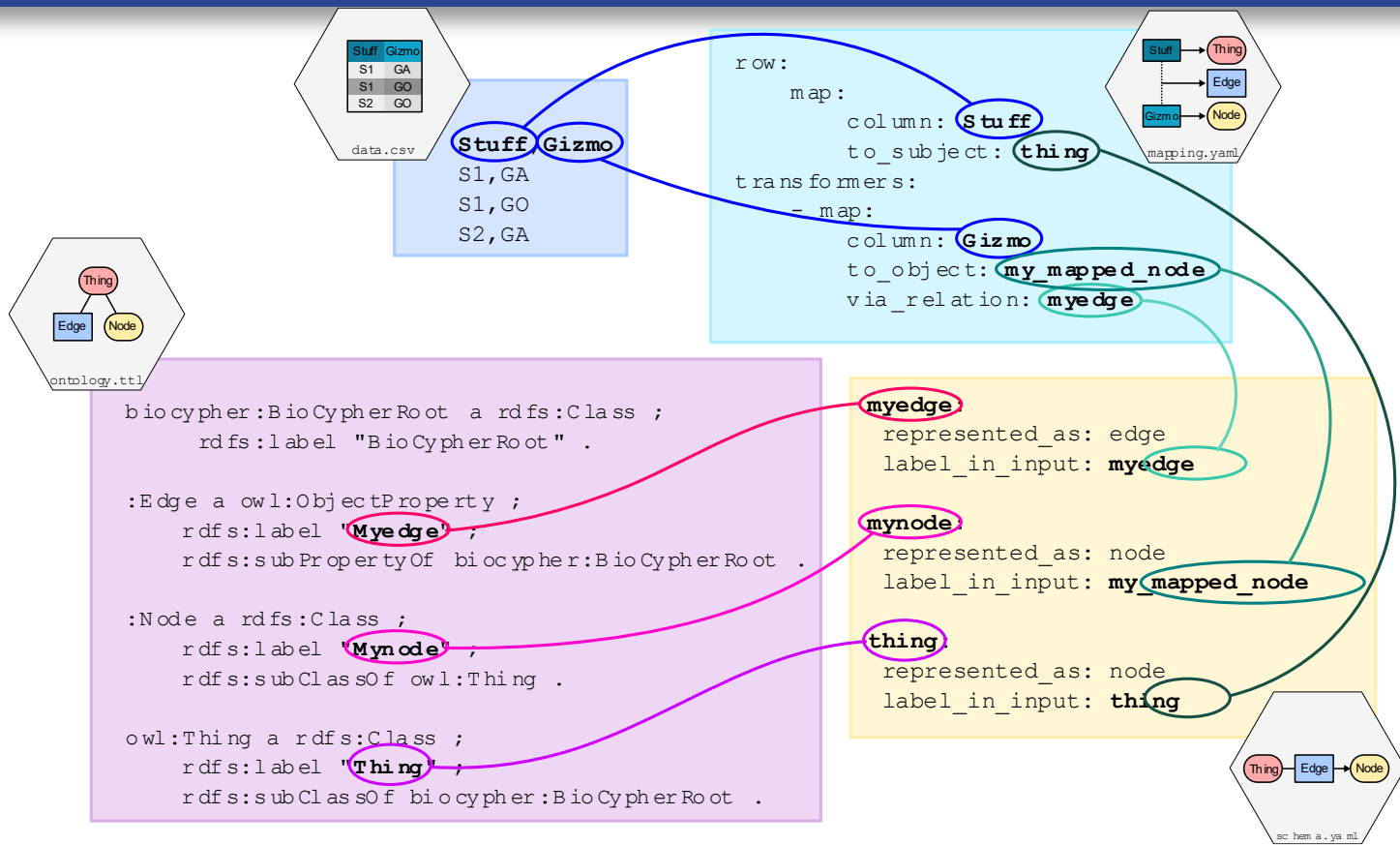
```
cd tests/demo/
```

```
ontoweave data.csv:mapping.yaml --biocypher-config config.yaml
--biocypher-schema schema.yaml
```

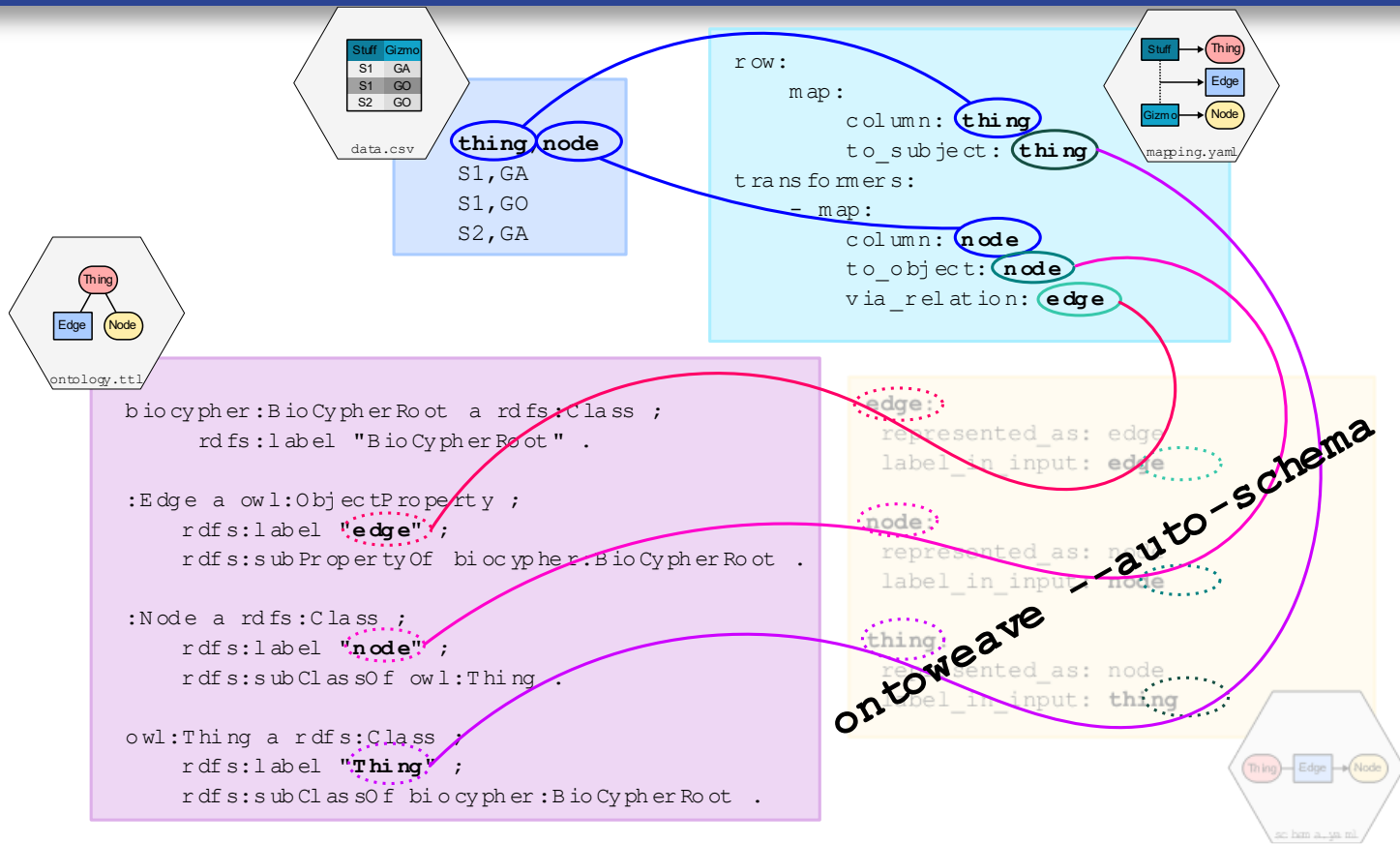
*Edit the mapping and load the JSON data.*



# Types Relationships



# auto-schema



# --auto-schema

```
$ ontoweave
 | data.csv:mapping.yaml
 | --biocypher-schema partial_schema.yaml
 | --auto-schema final_schema.yaml
```

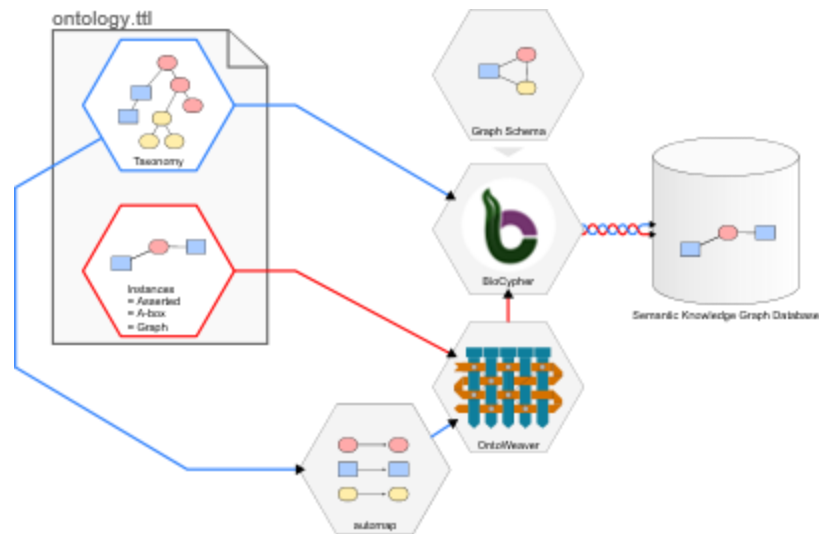
# Try it

```
cd tests/demo/
ontoweave data.csv:mapping.yaml --auto-schema final.yaml
```

```
cd ../tests/autoschema/
[...]
```

# Advanced OntoWeaver: fusion-enabled combination

```
$ ontoweave
| data_A1.csv:mapping_A.yaml
| data_A2.csv:mapping_A.yaml
| data_B/*.parquet:mapping_B.yaml
| ontology.ttl:automap
```



# Advanced OntoWeaver: mapping

## Advanced type management:

- Map the same item to several types depending on its value with *match*
- Use different columns to get the id and the types with *match\_type\_from\_column*
- Mapping different items to the same type with *final\_type*

# Advanced OntoWeaver: transformers

## User-defined transformers



`my_transformer.py :`

```
| class my_transformer(ontoweaver.base.Transformer):
| def __call__(self, row, i):
| [...]
```

`$ ontoweave --register my_transformer.py [...]`



`mapping.yaml :`

```
■ transformers:
| - my_transformer:
| [...]
```

# Other features

- **ontoweave**
  - `--import-script-run`
  - `--progress-bars`
  - `--auto-schema-overwrite`
- **automap**
  - Doc: Iterable input data > Web Ontology Data > OWL & automap

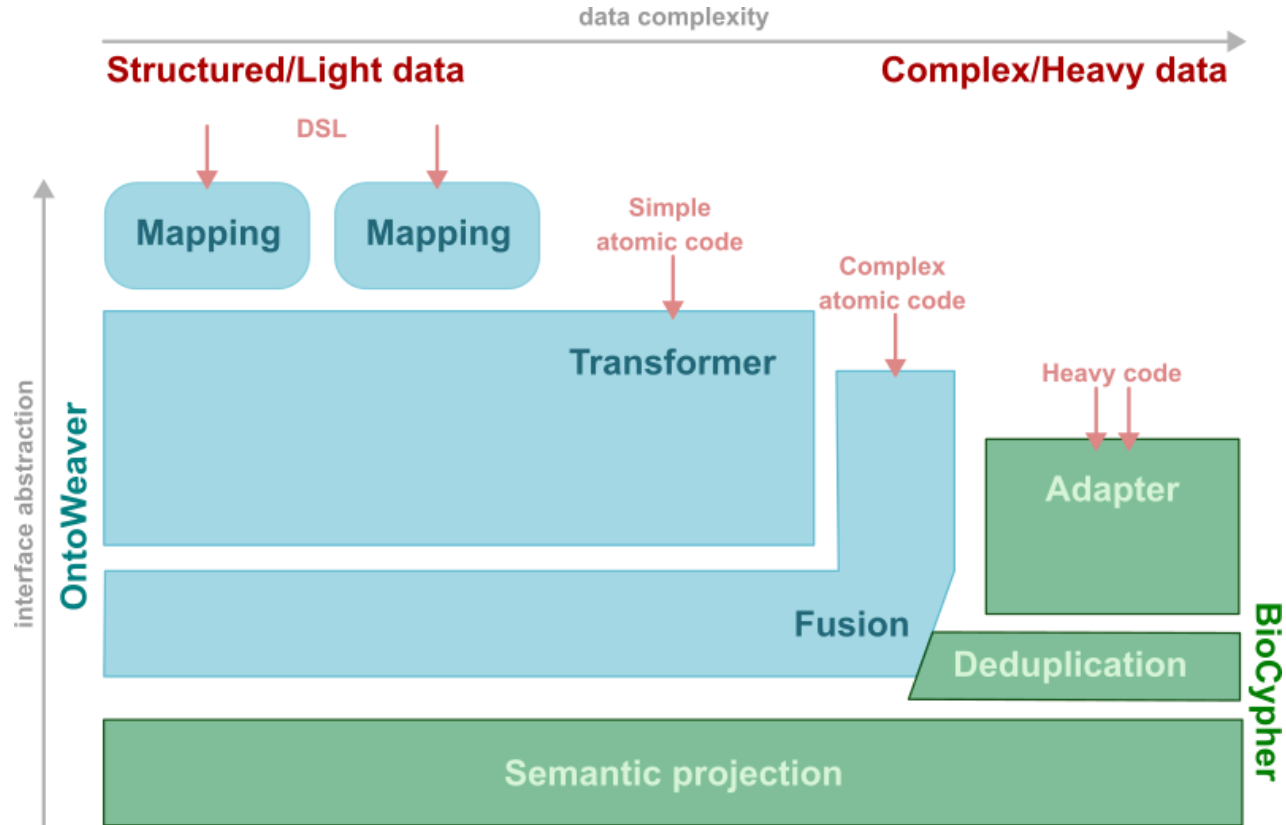


# Read the doc

<https://ontoweaver.readthedocs.io>



# OntoWeaver *or* raw BioCypher ?



# Suggested exercises

- If you already did a raw BioCypher adapter:
  - Try to do it with OntoWeaver
  - I'm interested in knowing how much lines of code you ended with
  - In how much time
- If you fancy code:
  - Try implementing your own transformer
    - Start from the “map” one in transformer.py
    - Share it!

# Thanks

Marko Baric, Taru Muranen, Altti Ilari Maarala, Sebastian Lobentanzer, Edwin Carreño, Jaana Oikkonen, Federico Bolelli, Vittorio Pipoli, Veli-Matti Isoviita, Ekaterina Gaydukova, Emanuele Quaglio, Inga Ulusoy, Johanna Hynninen



Codes: <https://github.com/ncodash>

Corresponding author: [johann.dreo@pasteur.fr](mailto:johann.dreo@pasteur.fr)

DECIDER 

Improving clinical decisions in cancer

OntoWeaver



OncodashKB

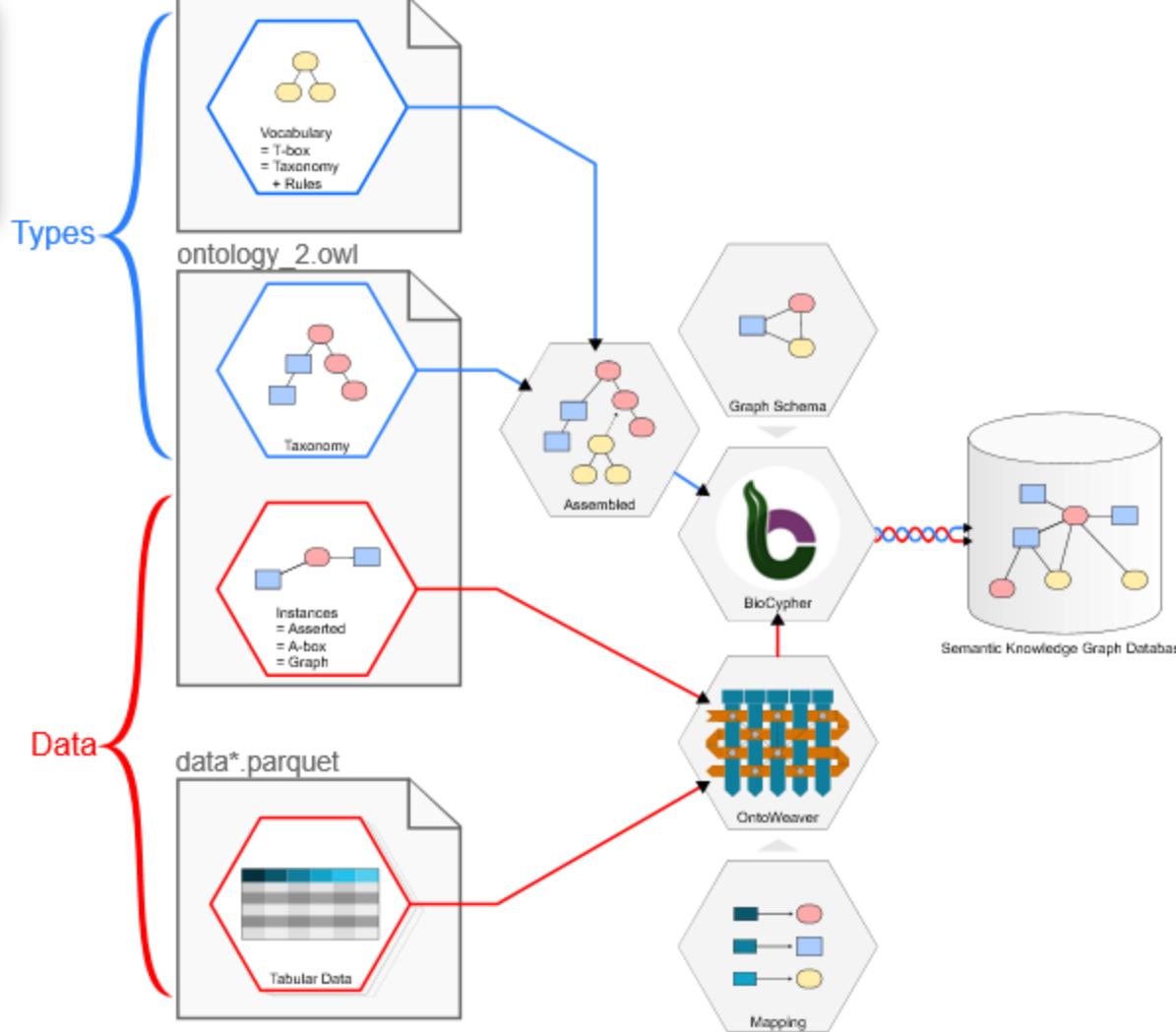


Fusion

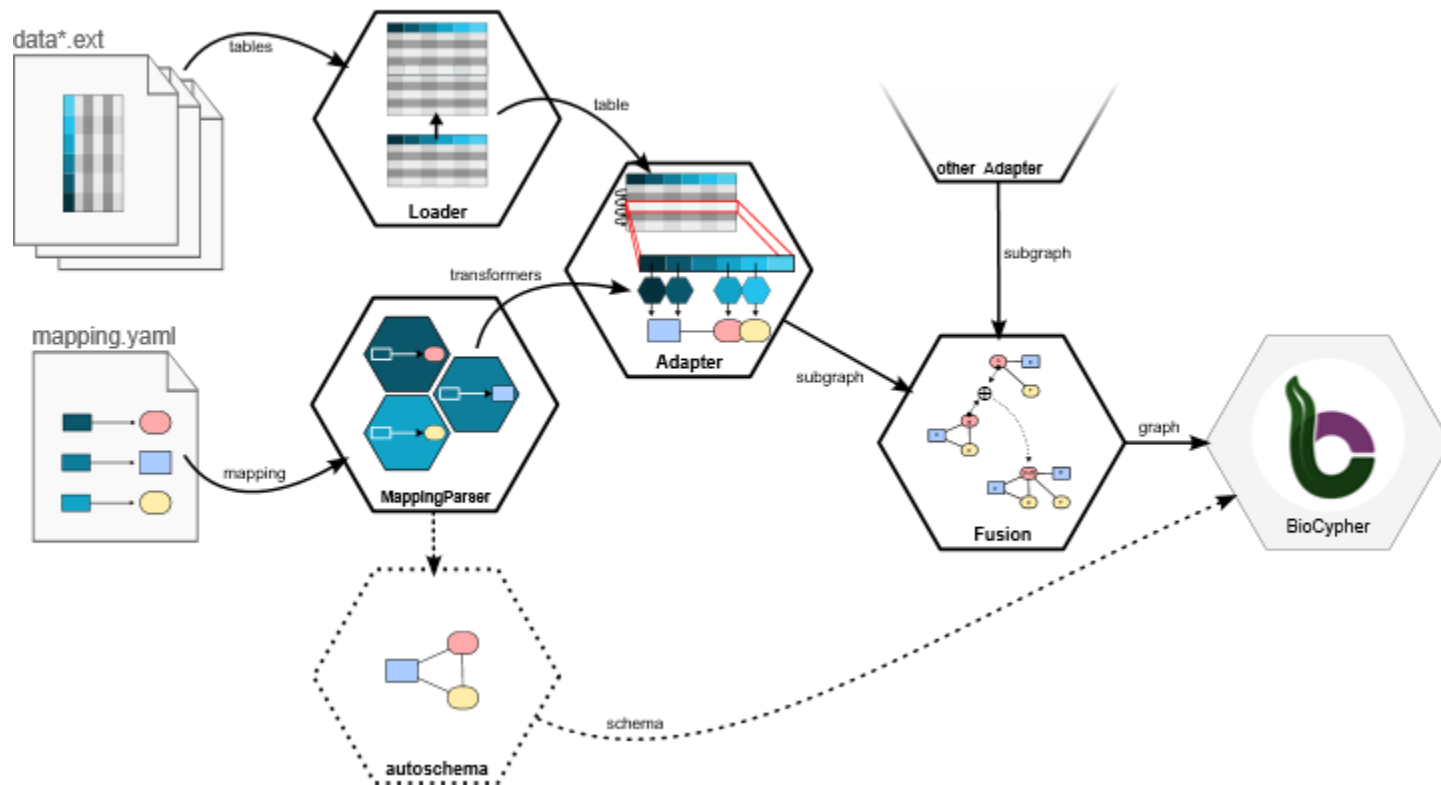




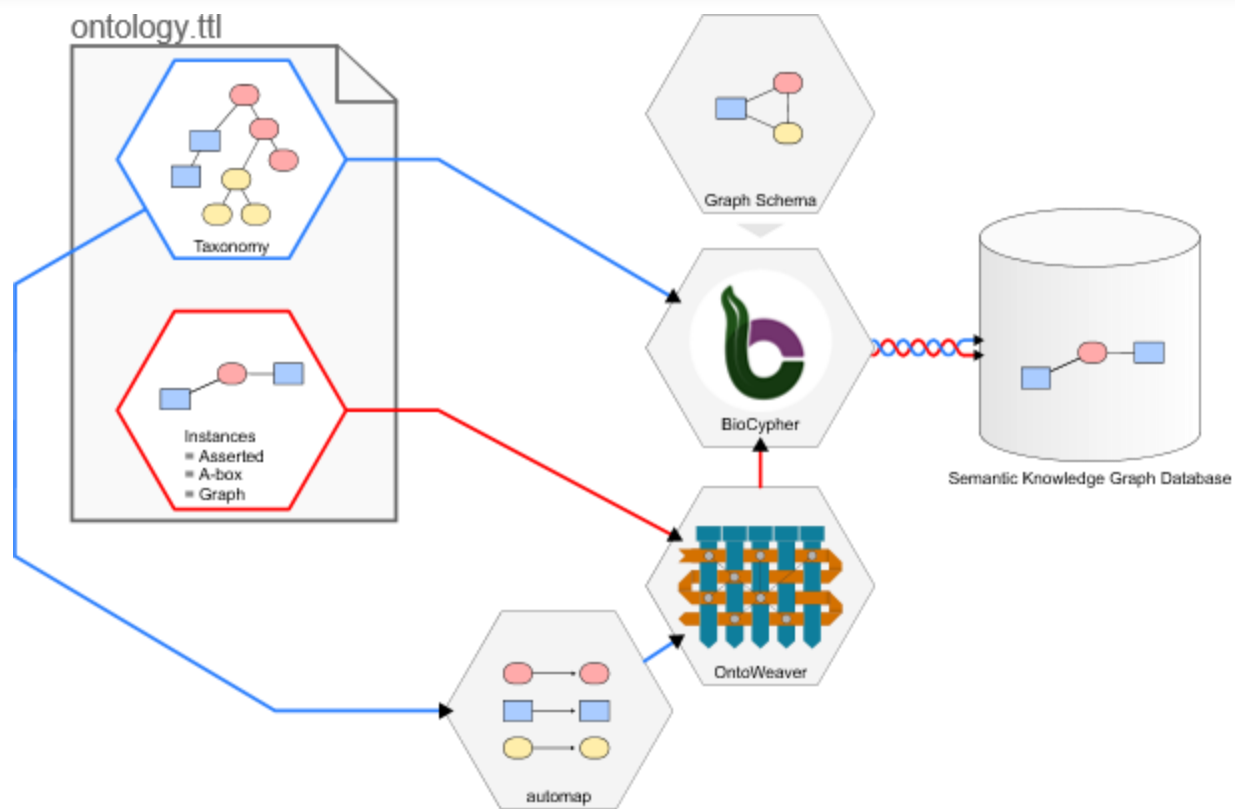
# ontology VS data



# OntoWeaver technical modules

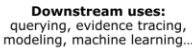


# automap

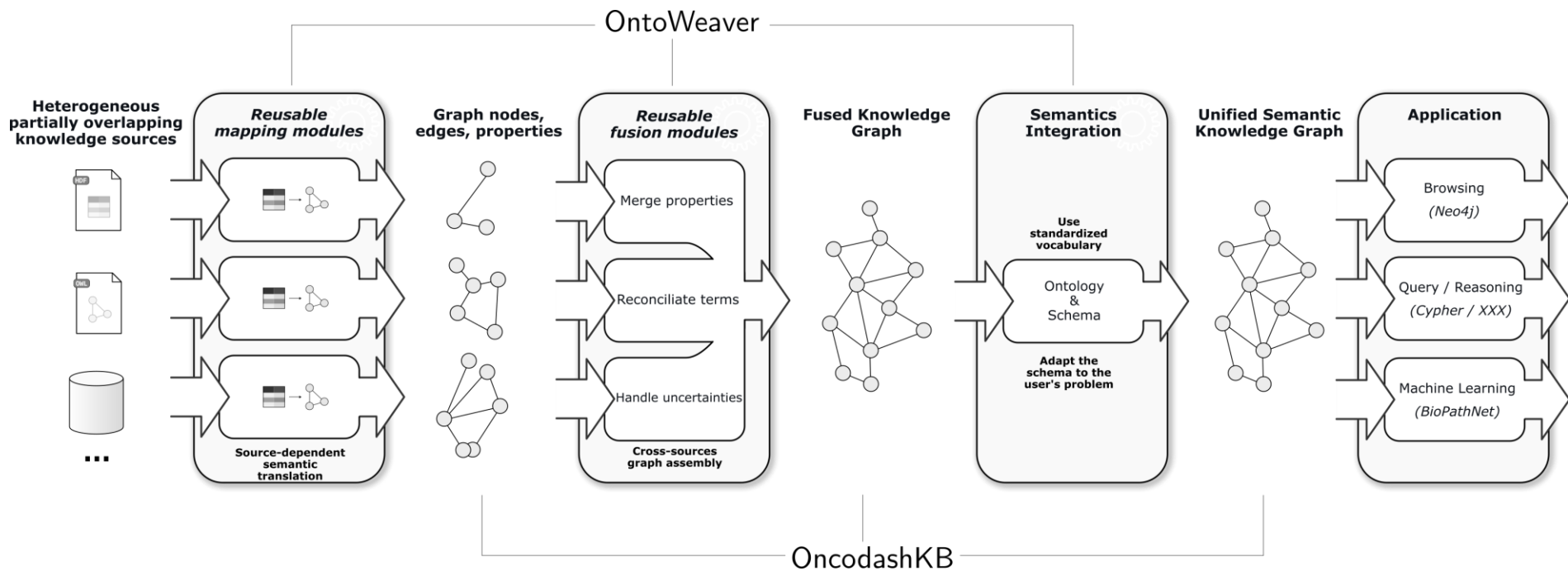




10 databases



# Reproducible Pipeline

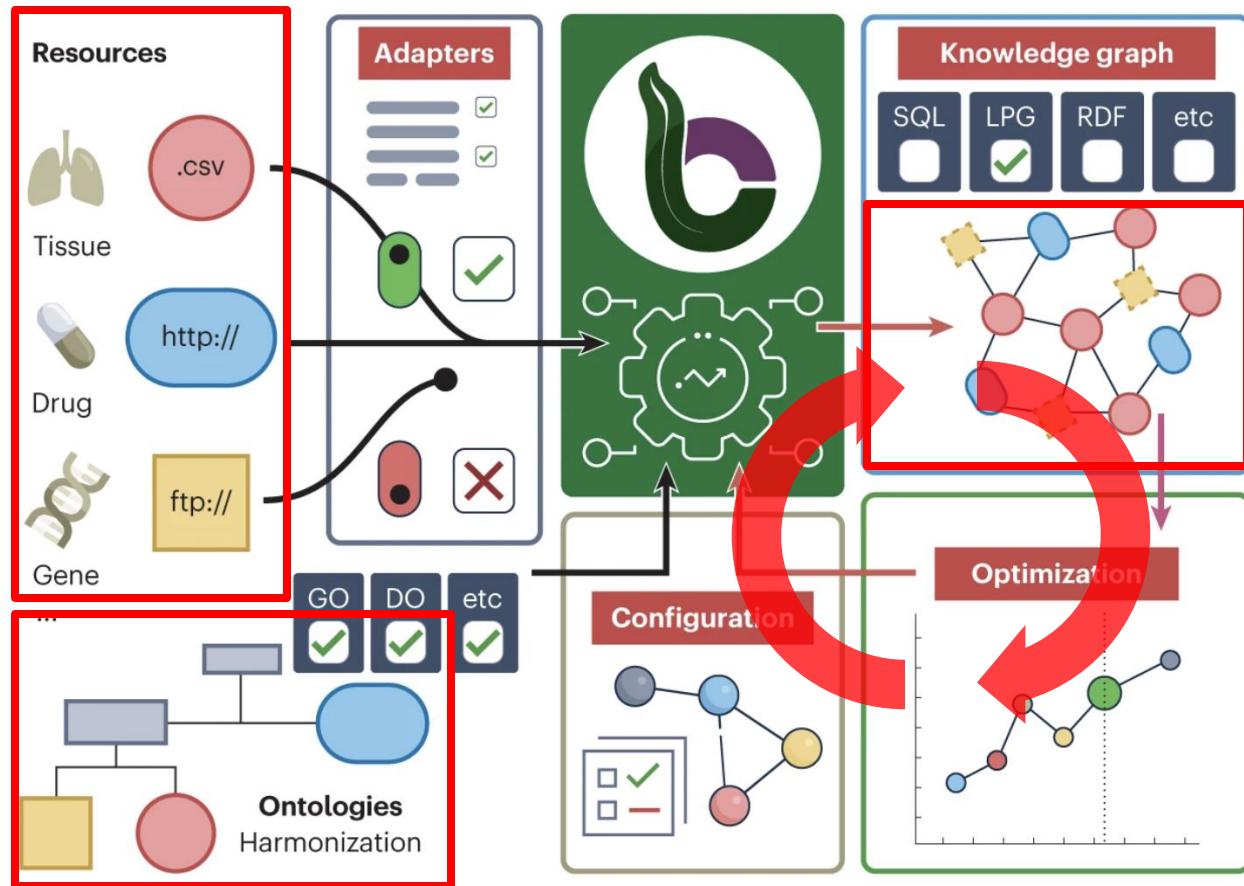


# biocypher

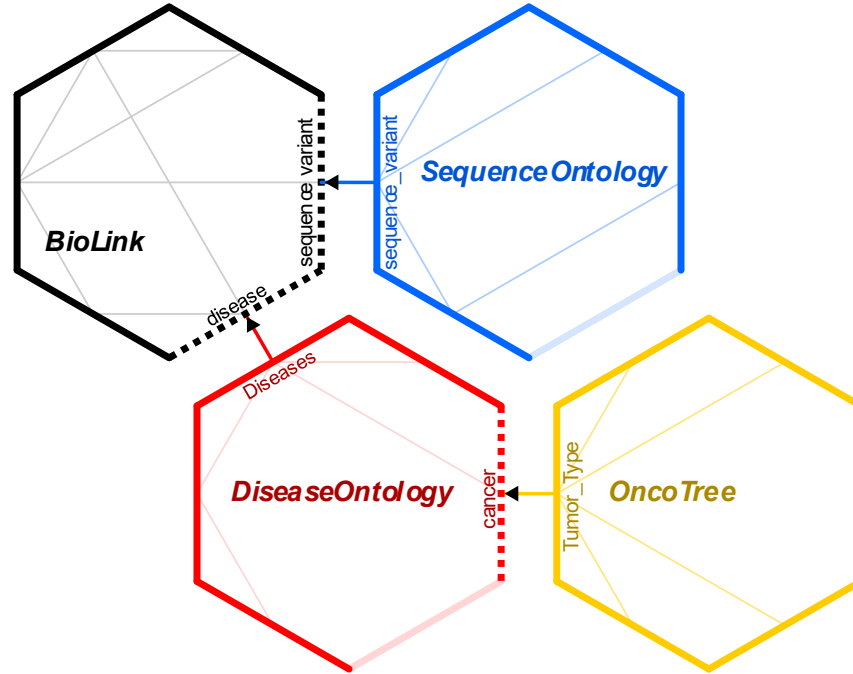
a unifying framework for  
biomedical knowledge graphs

## Democratizing knowledge representation with BioCypher

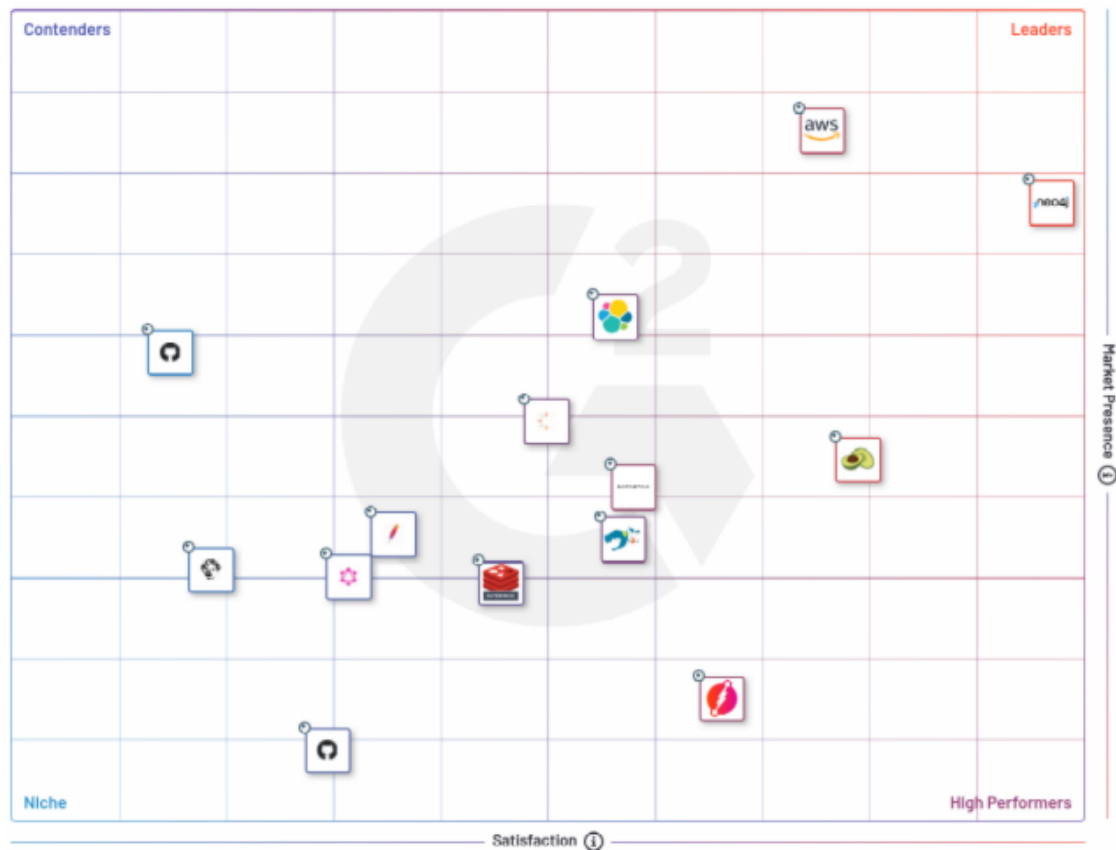
Sebastian Lobentanzer, ...,  
Johann Dreo, ..., Benno  
Schwikowski, ..., et al.  
*Nature Biotechnology*  
**volume 41**, pp. 1056–1059  
(2023).



# Ontologies Composition



# SKG engines



## Well established databases

- graph database
- + types hierarchy

<https://www.g2.com/categories/graph-databases>

# Ontology VS SKG?

